You encountered the tool for this job in chapter 4. If you look carefully at what you're trying to achieve with new threads, it's apparent that you're trying to calculate a result to return while allowing for the possibility that the code might throw an exception. This is *precisely* what the combination of `std::packaged_task` and `std::future` is designed for. If you rearrange your code to use `std::packaged_task`, you end up the following code.

Listing 8.3. A parallel version of `std::accumulate` using `std::packaged_task`

```
template<typename Iterator,typename T>
struct accumulate_block
{
    T operator()(Iterator first,Iterator last)      <-(1)
    {
        return std::accumulate(first,last,T());     <-(2)
    }
};
template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);

    if(!length)
        return init;

    unsigned long const min_per_thread=25;
    unsigned long const max_threads=
        (length+min_per_thread-1)/min_per_thread;

    unsigned long const hardware_threads=
        std::thread::hardware_concurrency();

    unsigned long const num_threads=
        std::min(hardware_threads!=0?hardware_threads:2,max_threads);
```